

ROS 2 二维机器人 运动规划实践

从仿真到定位、规划与控制

本书随代码逐章成长。每个例子都有明确的运行入口，
每个公式都能在对应该函数和实验中找到用途。

阅读方法 先运行、看见现象，再读公式和关键代码，最后完成一个小练习。
版本范围 当前实际实现范围以仓库进度表为准；本书只收录已实现内容。

主机 ROS 2 Lyrical / RViz2
独立算法 C++17, ROS 节点 C++20

2026 年 9 月

目录

第一章 环境搭建与第一个圆盘场景	1
1.1 先认识四个名字	1
1.2 确认主机环境	1
1.3 构建并启动	2
1.4 圆盘为什么用直径?	3
1.5 发布一次, 为什么晚启动也能看到?	3
1.6 动手练习与验收	4

第一章 环境搭建与第一个圆盘场景

本章目标 在主机 ROS 2 上编译两个小包，启动一个节点，在 RViz2 看见半径为 0.20 m 的圆盘。

先修 会打开终端、编辑文本，读懂基本 C++ 变量和函数。

1.1 先认识四个名字

节点 (node) 是执行计算并收发消息的程序。话题 (topic) 是带名字和类型的数据通道，例如本章的 `/visualization/robot`。包 (package) 组织源码、依赖和安装规则。工作空间把多个包放到一起编译；它不是新的 ROS 发行版。

本章只有一个发布节点和一个可视化程序：



RViz2 显示收到的数据，不负责机器人动力学。本章圆盘是一个静态图形。

1.2 确认主机环境

本课程验证平台是 Ubuntu 26.04 和 ROS 2 Lyrical。已有主机不用重装。打开 `bash`，先运行：

```
source /opt/ros/lyrical/setup.bash
printenv ROS_DISTRO
ros2 pkg prefix rviz2
colcon version-check --help
```

前三项应分别加载环境、输出 `lyrical`、找到 `RViz2`。最后一项只是检查 `colcon` 命令，后文构建才是实际工具链验收。

干净主机应先按 [ROS 官方安装步骤](#) 配置 UTF-8、Universe 与 `ros-apt-source`。已配好软件源的 Ubuntu 26.04 可安装 `ros-lyrical-desktop`、`ros-dev-tools`、`libeigen3-dev`。不要把其他 Ubuntu 版本与 Lyrical 的二进制包混用。

Lyrical 的 ROS 接口按 C++20 编译；本课程独立几何和数学代码采用 C++17。读者不必先学习 `concepts` 等高级特性。ROS 的 Python 包使用系统 Python，构建时显式指定 `/usr/bin/python3`。

1.3 构建并启动

以下路径是当前主机仓库位置；换机器后改成自己的路径。每个新终端都要先加载主机环境，再加载课程工作空间。

```
source /opt/ros/lyrical/setup.bash
cd /home/zdp/ForCodex/ros2_motion_planning_course/ros2_ws
colcon build --symlink-install --cmake-args \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DPython3_EXECUTABLE=/usr/bin/python3
source install/setup.bash
ros2 launch motion2d_bringup ch01.launch.py
```

motion2d 装算法和节点；motion2d_bringup 装配置、launch 与 RViz2 视图。--symlink-install 方便开发时编辑配置。参数只在启动时读取，所以修改 YAML 后重启节点。

RViz2 应显示浅色背景、0.5 m 网格和原点的青色圆盘，Fixed Frame 为 map。无图形界面可加 rviz:=false；这不能替代图形验收。

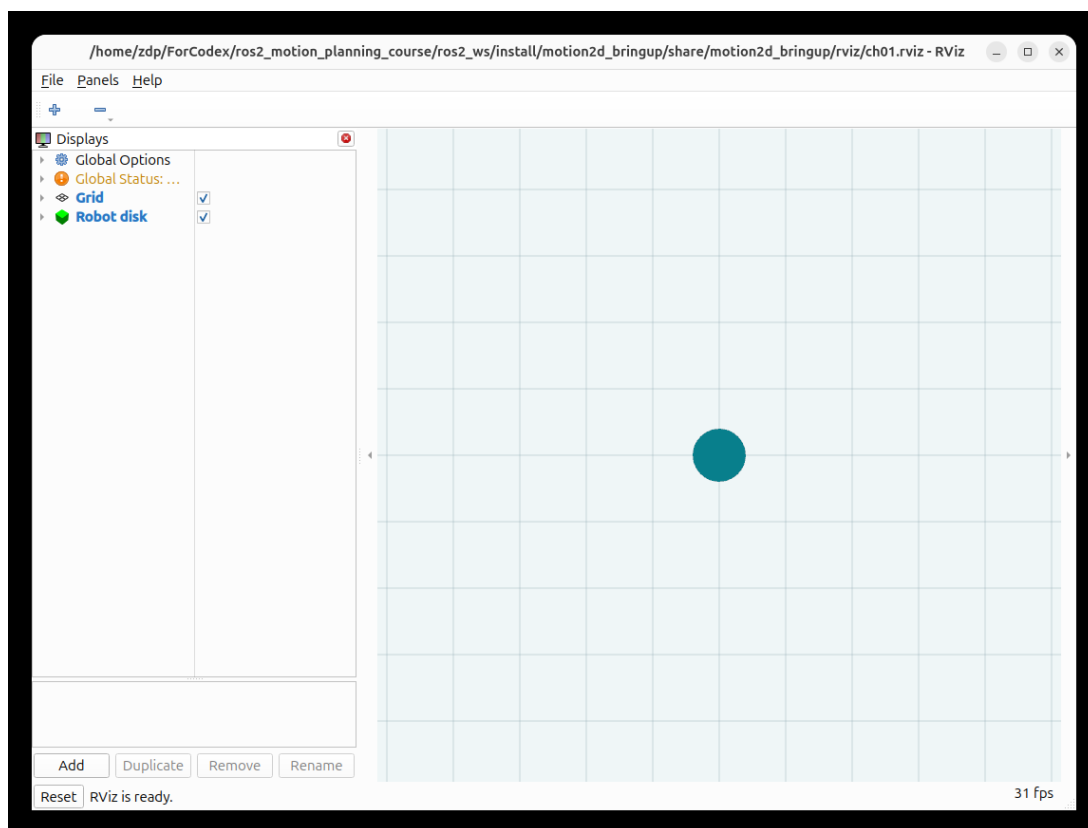


图 1.1: 主机实际运行画面。圆盘直径为 0.40 m，小于一个 0.5 m 网格。

此时 Global Status 可能提示尚无 TF 数据，因为本章还没有发布坐标树。Marker 已直接在 map 帧中，所以仍可正确显示；下一章建立 TF 后该提示消失。

1.4 圆盘为什么用直径？

用一个很薄的 CYLINDER 表示圆盘。Marker 的 scale 是物体的完整尺寸，因此

$$d_x = d_y = 2r, \quad r = 0.20 \text{ m} \Rightarrow d_x = d_y = 0.40 \text{ m}. \quad (1.1)$$

scale.z=0.05 只是显示厚度，后续平面碰撞只使用半径。单位四元数的 w 分量为 1；alpha 为 0 的图形完全透明。

下面直接读取实际源码的标记区域。修改代码后重新编译本书，摘录会同步更新：

```

33 visualization_msgs::msg::Marker disk;
34 disk.header.frame_id = "map";
35 disk.header.stamp = now();
36 disk.ns = "robot";
37 disk.id = 0;
38 disk.type = visualization_msgs::msg::Marker::CYLINDER;
39 disk.action = visualization_msgs::msg::Marker::ADD;
40 disk.pose.position.x = x;
41 disk.pose.position.y = y;
42 disk.pose.position.z = 0.025;
43 disk.pose.orientation.w = 1.0;
44 disk.scale.x = 2.0 * radius;
45 disk.scale.y = 2.0 * radius;
46 disk.scale.z = 0.05;
47 disk.color.r = 0.03F;
48 disk.color.g = 0.50F;
49 disk.color.b = 0.55F;
50 disk.color.a = 1.0F;
51 //

```

1.5 发布一次，为什么晚启动也能看到？

本节点使用 reliable + transient-local QoS，保留最后一条 Marker。RViz2 订阅同样配置持久性，晚启动时可取到保留的消息。节点退出后，不应假设消息仍会被保留。

第二个终端加载环境，进入工作空间后运行：

```

ros2 topic echo /visualization/robot --once \
--qos-durability transient_local
/usr/bin/python3 ../scripts/check_ch01.py

```

检查程序确实作为晚加入的订阅者接收消息，并核对帧、类型与默认尺寸。此时消息 stamp 用系统时间；下一章再引入仿真时钟。

1.6 动手练习与验收

1. **理解：**节点与话题的区别是什么？发布者先启动、RViz2 后启动时，为什么仍能收到圆盘？
2. **代码：**完成 `exercises/ch01/starter/marker_scale.hpp` 中的 `EXERCISE(ch01-1)`。检查方法、分级提示和独立参考答案位于同目录。
3. **实验：**在 `ch01.yaml` 中把半径改成 0.35 m、x 改为 1.0 m，重启。先预测消息和网格上的尺寸，再观察；检查默认配置前恢复参数。

应看到直径 0.70 m、圆心 (1,0) 的圆盘。若图形不见了，依次检查 Fixed Frame、话题、alpha 与 QoS。若找不到包，检查是否加载课程的 install 环境。

独立编译练习

练习的输入是半径，输出是 Marker 的 x/y 尺寸。下面的 starter 故意保留一个需要你修改的返回值；它没有被链接到正式演示中。

```
#pragma once

/** @brief Return Marker x/y size (m) for a disk of positive radius (m). */
inline double markerDiameter(double radius)
{
    // EXERCISE(ch01-1): RViz's cylinder scale describes its full width.
    return radius;
}
```

从仓库根目录运行：

```
g++ -std=c++17 -Wall -Wextra -Iexercises/ch01/starter \
    exercises/ch01/check.cpp -o /tmp/ch01_exercise
/tmp/ch01_exercise
```

未完成时检查输出 FAIL；改对后输出 PASS。检查包含两个半径，避免只为一个例子填写常数。若卡住，先看同目录的 hints，再读 solutions 的解释；把编译命令中的 starter 换成 solutions 即可验证参考解。

本章完成后，你能解释图形由哪个节点发布、消息中哪些量决定尺寸，以及如何让另一个程序接收这条消息。